# *C/C++-Kurs für Kinder ab der 5. Klasse*

*Jürgen "George" Sawinski*

*3. September 2026*

Copyright © 2026 Jürgen “George” Sawinski

VERÖFFENTLICHT VOM AUTOR

https://github.com/jsawinski/twin-c-course

Licensed under the MIT License

Copyright (c) 2026 Jürgen “George” Sawinski

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Dieser Kurs ist gedacht für Kinder ab Ende der Grundschule (ca. 5. Klasse), die Begeisterung für Mathematik und Computer mitbringen und den Drang haben, programmieren zu lernen.

Der erste Teil des Kurses beschäftigt sich damit, alle grundlegenden Kenntnisse zu erwerben, um eine Konsolen-basierte Regensimulation in C zu programmieren, was auch mathematische und physikalische Grundlagen beinhaltet.

Der zweite Teil des Kurses wird das Ziel haben, das Spiel Tetris® in C++ umzusetzen, wobei das Hauptaugenmerk dabei auf die Grundlagen Objekt-orientierter Programmierung gelegt wird.

# *Inhaltsverzeichnis*


**I Die Programmiersprache C** **9**

**1 Ein paar Grundlagen** **11**
1.1 Das Binärsystem . . . . . 11
1.2 Logikgatter — die Bausteine des Prozessors . . . . . 13
1.3 Bitweise Operationen in C . . . . . 15

**2 Erste Schritte in C** **17**
2.1 Programm: Hallo Welt! . . . . . 17
2.2 Programm: Zahlen . . . . . 19
2.3 Programm: Fibonacci . . . . . 21
2.4 Programm: Reihen . . . . . 23
2.5 Programm: Lauflicht . . . . . 26

**3 Etwas Physik** **27**

**4 Regensimulation in C** **29**
4.1 Warnungen des Compilers . . . . . 29

**II Die Programmiersprache C++** **31**

**III Anhang** **33**

**A Software** **35**

**B Leitfaden** **37**
B.1 Tastenkombinationen . . . . . 37
B.1.1 Sonderzeichen . . . . . 38
B.1.2 Referenz . . . . . 38
B.2 C . . . . . 39
B.2.1 Grundlagen . . . . . 39
B.2.2 Datentypen . . . . . 39
B.2.3 Strukturen . . . . . 41

B.2.4 Anweisungsblock . . . . . 41
B.2.5 Kontrollstrukturen . . . . . 42
B.2.6 Schleifen . . . . . 44
B.2.7 Funktionen . . . . . 45
B.2.8 Namen . . . . . 46
B.2.9 Referenz . . . . . 47

**C Lösungen zu den Übungen** **51**

**Index** **57**

## *Vorwort*

Dieser Kurs ist aus der gemeinsamen Arbeit des Autors mit seinen Söhnen entstanden. Die Idee, einen C/C++-Kurs für Kinder zu schreiben, kam von den Jungs selbst, die unbedingt programmieren lernen wollten um ihre eigenen Spiele zu entwickeln zu können.

Die Mitschrift ist in erster Linie für Kinder gedacht, die den Kurs mit einer Begleitperson zusammen durchlaufen und dient als Leitfaden sowie als Nachschlagewerk für die behandelten Themen.

Der Autor hat die Inhalte erarbeitet, erklärt und in LaTeX festgehalten. Bei der Strukturierung des Materials und der Erstellung einiger LaTeX-Makros wurde er durch KI-gestützte Werkzeuge (**DeepSeek** und **GitHub Copilot**) unterstützt.

DeepSeek lieferte Vorschläge zur Gliederung, zur didaktischen Aufbereitung und zu technischen Umsetzungen in LaTeX. Github Copilot wurde beim Schreiben von LaTeX-Code und bei der Erstellung von C/C++-Codebeispielen eingesetzt. Alle Vorschläge wurden vom Autor geprüft, angepasst und auf die Bedürfnisse seiner Kinder zugeschnitten.

*– Jürgen "George" Sawinski, 3. September 2026*

## *Lizenz und Haftungsausschluss*

Alle Inhalte dieses Kurses wurden vom Autor selbst erstellt und sind, soweit nicht anders gekennzeichnet, frei von urheberrechtlich geschützten Materialien Dritter. Die Beispiele und Übungen sind so gestaltet, dass sie die Lernziele unterstützen und den Kindern helfen, die Konzepte zu verstehen und anzuwenden.

Dieses Werk wird unter der **MIT-Lizenz** veröffentlicht. Das bedeutet: jeder darf es kopieren, verändern, weitergeben und für eigene Zwecke nutzen — auch kommerziell. Die einzige Bedingung ist, dass der Autor (Jürgen "George" Sawinski) in der Quellenangabe genannt wird.

**Haftungsausschluss:** Dieser Kurs wurde mit größter Sorgfalt erstellt. Dennoch übernimmt der Autor keine Gewähr für die Richtigkeit, Vollständigkeit oder Aktualität der Inhalte. Die Nutzung der Inhalte erfolgt auf eigene Verantwortung. Der Autor haftet nicht für Schäden, die aus der Nutzung oder der Unfähigkeit zur Nutzung der Inhalte entstehen.

**Projekt-Webseite:** `https://github.com/jsawinski/twin-c-course`

# Teil I

# Die Programmiersprache C

# 1
# *Ein paar Grundlagen*

## *1.1 Das Binärsystem*

Computer rechnen im sogenannten Binärsystem, sie kennen nur 0 und 1. Um größere Zahlen darstellen zu können, werden mehrere dieser Nullen und Einsen, genannt "**Bit**" aneinandergereit. Am gebräuchlichsten dabei ist das "**Byte**" (siehe Abb. 1.1), das aus 8 Bit besteht.
Jedem Bit wird nun ein Wert zugeordnet. Das erfolgt mathematisch gemäß:

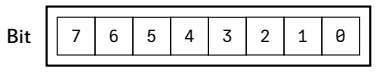



Abbildung 1.1: Ein Byte mit den Bit-Positionen 0 bis 7.

$$2^b$$

wobei $b$ die Nummer des Bits ist. Das ist die sogenannte "Potenz", eine mathematische Operation[1] die sagt, wie oft eine Zahl mit sich selbst multipliziert wird. In diesem Fall wird die "2" $b$-mal mit sich selbst multipliziert, zum Beispiel:

$$\begin{aligned} 2^1 &= 2 && = 2 \\ 2^2 &= 2 \cdot 2 && = 4 \\ 2^3 &= 2 \cdot 2 \cdot 2 && = 8 \end{aligned}$$

Interessant ist nun die Frage, was $2^0$ ist. Wenn Ihr genau hinschaut, dann merkt Ihr, dass, um in die nächste Zeile zu kommen, Ihr das Ergebnis mit 2 multipliziern müsst. Umgekehrt genauso: um in die vorhergehende Zeile zu kommen, muss man durch 2 teilen. Wenn also $2^1$ zwei ist und man, um zu $2^0$ zu kommen durch zwei Teilen muss, dann ergibt sich

$$2^0 = 1\,.$$

Übrigens, jede Zahl hoch 0 hat als Ergebnis die 1 (wir kommen später darauf zurück).[2]

Um auf das **Byte** zurückzukommen, heißt dass, dass den **Bits** jeweils die Werte 1 bis 128 zugeordnet sind (siehe Abb. 1.1).

[1] In der Schule habt Ihr wahrscheinlich den Begriff der Grundrechenarten (oder Grundrechnungsarten) kennengelernt. Der Begriff "Operation" bedeutet *Ausführungshandlung*, also ein Vorgang, bei dem aus einem oder mehreren Werten ein neuer Wert berechnet wird - oder in anderen Worten eine *Rechnungsart*. Die **Zeichen**, die diese Operationen beschreiben (wie **+**, **-**, **&**, **|**), nennt man **Operatoren**. Die Werte, mit denen eine Operation arbeitet, heißen **Operanden**. Zum Beispiel sind die Grundrechenarten, die Ihr in der Schule gelernt habt, also Addition, Subtraktion, Multiplikation und Division (Plus, Minus, Mal, Geteilt), sogenannte **arithmetische Operationen**.

[2] Es gibt ein Ausnahme und das ist $\mathbf{0^0}$. Wenn Ihr diese Rechnung auf dem Computer ausführt, dann erhaltet Ihr als Ergebnis höchstwahrscheinlich auch die **1**. Die Mathematik dahinter kommt allerdings nicht zu einem eindeutigen Ergebnis.

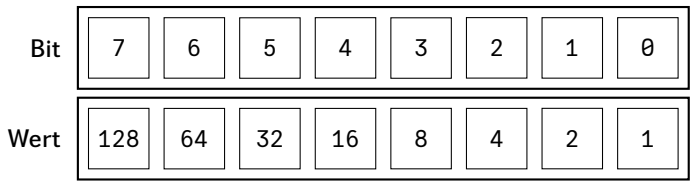



Abbildung 1.2: Ein Byte mit Bit-Positionen und den zugehörigen Werten ($2^b$).

Damit können wir nun die Zahlen 0 bis 255 darstellen, z.B.:

$$00000000 \rightarrow 0\,,$$
$$00000011 \rightarrow 3\,,$$
$$01011000 \rightarrow 88\,,$$
$$11111111 \rightarrow 255\,.$$

Das Addieren funktioniert nun genauso, wie Ihr es aus der Schule, in der natürlich das Zehnersystem verwendet wird, gelernt habt. Beispiel:

```
  00100111       39
+ 00000110     +  6
     11          1     (Übertrag)
 ---------      ---
  00101101       45
```

Entsprechend lässt sich auch die Multiplikation durchführen:

```
00001010·00101010 =     10·42
        ..|.|.|.
  00000000.|.|.|.
   00000000|.|.|.
    00001010.|.|.
     00000000|.|.
      00001010.|.
       00000000|.
        00001010.
         00000000
         1 1             (Übertrag)
 ---------------
       110100100        420
```

Wenn Ihr genau hinseht, dann bemerkt Ihr, dass wir zwei Bytes multipliziert haben, aber das Ergebnis benötigt 9 Bits, also ein Bit mehr als ein Byte. Das führt zu dem Problem, dass, wenn Ihr auf einem Computer diese Multiplikation durchführt, das höchste Bit (die letzte "1" links) abgeschnitten wird und das Ergebnis, dass der Computer euch liefert, 164 ist. Das gleiche passiert bei einem Byte, wenn Ihr zu 255 die 1 dazuzählt, das Ergebnis ist 0 (in der Fachsprache als **Überlauf** bezeichnet).

Computer können auch negative Zahlen darstellen. Dazu nützt man aus, dass, wenn 255 + 1 zum Ergebnis 0 hat, dann hat 0 – 1 als Ergebnis –1, binär also 11111111. Dies nennt man das "Zweierkomplement" (den Begriff müsst Ihr euch natürlich nicht merken). Wichtig ist: dass höchste

Bit zeigt, dass es sich um eine negative Zahl handelt. Damit können dann mit einem Byte die Zahlen -128 bis 127 dargestellt werden:

```
10000000        -128
10000001        -127
...
11111111        -  1
00000000           0
00000001           1
...
01111111         127
```

Wie wird nun aus einer positiven Zahl eine Negative? Das passiert, indem man alle Bits invertiert (das heißt, aus 0 wird 1 und aus 1 eine 0) und 1 addiert:

```
(-127)  10000001  ->  01111110
                     +00000001
                      --------
                      01111111     (127)
```

Das funktioniert auch umgekehrt, also aus einer negativen Zahl eine positive zu machen:

```
(127)   01111111  ->  10000000
                     +00000001
                      --------
                      10000001     (-127)
```

## *1.2 Logikgatter — die Bausteine des Prozessors*

Im vorhergehenden Abschnitt haben wir gesehen, dass Computer im Binärsystem rechnen. Das bedeutet, dass sie nur mit den Zahlen 0 und 1 arbeiten. Aber wie kann ein Computer damit rechnen? Wie kann er zum Beispiel die Zahl 2 mit der Zahl 3 multiplizieren? Bevor wir das verstehen, müssen wir uns anschauen, wie Computer aufgebaut sind: die zentrale Recheneinheit (CPU) besteht aus vielen kleinen Schaltungen, die mit nur zwei Zuständen arbeiten: **0** und **1**. Diese Schaltungen nennt man **Logikgatter**. Die wichtigsten sind:

*UND-Gatter* [3]

[3] Englisch: **and**

Das UND-Gatter liefert nur dann eine 1, wenn beide Eingänge 1 sind.

| A | B | A and B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

*ODER-Gatter* [4]

[4] Englisch: **or**

Das ODER-Gatter liefert eine 1, wenn mindestens einer der Eingänge 1 ist.

| A | B | A or B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

*XOR-Gatter (exklusives ODER)* [5]

[5] Englisch (kurz): **xor**

Das XOR-Gatter liefert eine 1, wenn genau einer der Eingänge 1 ist.

| A | B | A xor B |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

*NICHT-Gatter* [6]

[6] Englisch: **not**

Das NICHT-Gatter invertiert den Eingang: aus 0 wird 1, aus 1 wird 0.

| A | not A |
|---|---|
| 0 | 1 |
| 1 | 0 |

Aus der Kombination dieser Logikgatter kann man dann auch komplexere Schaltungen bauen, die zum Beispiel alle Grundrechenarten (Addition, Subtraktion, Multiplikation und Division) von zwei Zahlen durchführen können.

**Übung 1a:** Wir haben oben die Addition von zwei binären Zahlen durchgeführt. Für die erste Stelle (ganz rechts) haben wir die beiden Bits addiert und das Ergebnis, aber auch den Übertrag notiert. Finde heraus, wie man die Addition der beiden Bits und den Übertrag mit den binären Operationen `AND`, `OR` und `XOR` berechnen kann.

*Hinweise*: Zur Lösung der Aufgabe ist es sinnvoll, eine Wahrheitstabelle für die Addition von zwei Bits und den Übertrag zu erstellen:

| A | B | S | C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| ... | ... | ... | ... |

Hier steht das "S" steht für die Summe (Englisch "sum") und das "C" für den Übertrag (Englisch "carry").

**(Lösung S. 51)**

## *1.3 Bitweise Operationen in C*

Die Logikgatter, die wir im vorhergehenden Abschnitt kennengelernt haben, sind die Bausteine, aus denen Computer aufgebaut sind. In C und C++, sowie vielen anderer Programmiersprachen, stehen diese als sogenannte **bitweise Operationen** zur Verfügung. Mit ihnen lassen sich die gleichen Operationen durchführen, wie mit den Logikgattern. Das bedeutet, dass Ihr mit diesen Operatoren auf den einzelnen Bits einer Zahl arbeiten könnt. Die Liste der bitweisen Operatoren in C und C++ ist:

*& (and)*
Beispiel: `1101 & 1011 = 1001`

*| (or)*
Beispiel: `1101 | 1011 = 1111`

*^ (xor)*
Beispiel: `1101 ^ 1011 = 0110`

*~ (not)*
Beispiel: `~1101 = 0010`

Darüber hinaus gibt es noch die sogenannten **Schiebe-Operatoren**[7], mit denen Ihr die Bits einer Zahl nach links[8] oder rechts[9] verschieben könnt. Das ist zum Beispiel nützlich, wenn Ihr eine Zahl mit 2 multiplizieren oder durch 2 teilen wollt. Die Shift-Operatoren sind:

*<< (shift left)*
Beispiel: `00000011 << 2 = 00001100`

*>> (shift right)*
Beispiel: `11000101 >> 2 = 00110001`

Vermutlich werdet Ihr Euch jetzt fragen, wozu diese bitweisen Operationen überhaupt gut sind. Es gibt eine Vielzahl von Anwendungsmöglichkeiten, zum Beispiel werden wir später sehen, dass sich damit sehr einfach überprüfen lässt, ob eine Zahl gerade oder ungerade ist. Darüber hinaus kann auch eine Vielzahl von Ein- und Ausgabekanälen eines Computers sehr einfach angesteuert werden, ohne dass dafür komplexe Berechnungen notwendig wären. Es gibt aber viele Algorithmen[10], die sich mit nur mit bitweisen Operationen gut umsetzen lassen.

**Übung 1b:** Wir haben oben die Addition von zwei binären Zahlen an folgendem Beispiel durchgeführt:

```
  00100111      39
+ 00000110    +  6
    11          1     (Übertrag)
```

[7] Englisch: **shift**

[8] Englisch: **left**

[9] Englisch: **right**

[10] Ein Algorithmus ist eine *genaue, endliche Abfolge von eindeutigen Anweisungen* zur Lösung eines Problems. Ein Kochrezept, eine Bauanleitung oder die Anleitung für ein Videospiel sind alles Algorithmen – in der Informatik sind sie die *Baupläne* für Programme.

```
---------      ---
 00101101       45
```

Aus der letzten Übung wisst Ihr bereits, dass Ihr für das 0-te Bit (ganz rechts), aus den Zahlen $A$ und $B$ durch **xor** die Summe und durch **and** den Übertrag berechnen könnt. In Form eines Algorithmus lässt sich die das folgendermaßen darstellen:

```
S = A xor B
C = A and B
```

Versucht nun herauszufinden, welchen weiteren Schritte notwendig sind, um die Summe von $A + B$ zu berechnen.

**(Lösung S. 51)**

# 2
# Erste Schritte in C

## 2.1 Programm: Hallo Welt!

In alter Tradition ist das erste Programm, das man zu sehen bekommt, wenn man eine neue Programmiersprache lernt, gewöhnlich das "Hallo Welt!" Programm.[1] Da wir uns zunächst auf die Programmiersprache "C" beschränken, ist das:

[1] siehe Wikipedia 2003b

```c
/* Das "Hallo Welt"-Programm. */
#include <stdio.h>

int main() {
    printf("Hallo Welt!\n");
    return 0;
}
```

*Erklärung:*

`/* Das "Hallo Welt"-Programm. */`

Das ist ein **Kommentar**. Der Computer ignoriert ihn völlig. Er ist nur für uns Menschen da, um den Code zu erklären.

In C gibt es zwei Arten von Kommentaren:

- `/* ... */` — für längere Kommentare über mehrere Zeilen.
- `// ...` — für kurze Kommentare in einer Zeile.

`#include <stdio.h>`

Das ist eine **Header-Datei** (auf Deutsch auch *Kopfdatei*).

`stdio.h` steht für *Standard Input Output*. Sie enthält Informationen über Funktionen wie `printf`, die wir später benutzen wollen.

**Merke:** Wenn wir eine Funktion benutzen wollen, die wir nicht selbst geschrieben haben, müssen wir die dazugehörige Header-Datei einbinden.

```
int main() {
```

Das ist die **Hauptfunktion** (englisch: *main function*).

Jedes C-Programm startet hier. Der Computer führt den Code in den geschweiften Klammern `{ ... }` aus.

`int` bedeutet, dass die Funktion eine ganze Zahl (Integer) zurückgibt — das ist der Rückgabewert, der weiter unten kommt.

```
printf("Hallo Welt!\n");
```

Das ist die **Ausgabe**.

`printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

**Wichtig:** Jede Anweisung in C endet mit einem Semikolon `;`.

```
return 0;
```

Das ist die **Rückgabe**.

Sie sagt dem Betriebssystem: *"Alles ist gut gelaufen."*

`0` bedeutet in der Regel *"kein Fehler"*. Wenn ein Programm mit `return 0;` endet, weiß der Computer, dass das Programm erfolgreich war.

```
}
```

Die schließende geschweifte Klammer beendet die `main`-Funktion. Alles zwischen { und } gehört zur Funktion.

*Merke:*

- Ein C-Programm beginnt immer in der `main()`-Funktion.
- `printf()` schreibt Text auf den Bildschirm.
- `\n` macht einen Zeilenumbruch.
- `return 0;` sagt dem Computer: "Alles OK."
- Jede Anweisung endet mit `;`.

## 2.2 Programm: Zahlen

Um einen Computer dazu zu bringen, etwas zu tun, müssen wir ihm sagen, was er tun soll. Dazu brauchen wir Funktionen und Variablen. Eine **Variable** ist ein Platzhalter für einen Wert. Wir können ihr einen Namen geben und ihr einen Wert zuweisen. Beispiel: Wir wollen eine Variable namens `zahl` erstellen und ihr den Wert 42 zuweisen. In C sieht das so aus:

```c
/* Deklaration einer Zahl. */
#include <stdio.h>

int main() {
  int zahl = 42;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

*Erklärung:*

*`int zahl = 42;`*

Das ist die **Deklaration**[2] (Festlegung des Namens und um was es sich handelt) und **Initialisierung**[3] (die erste Zuweisung eines Wertes) einer Variablen.

`int` ist der Datentyp, in diesem Fall eine ganze Zahl (Integer).
`zahl` ist der Name der Variable.[4]
`42` ist der Wert, der der Variable zugewiesen wird.

Es ist auch möglich, eine Variable folgendermaßen zu deklarieren:

```c
  int zahl; // Deklaration ohne Initialisierung
```

und ihr zu einem späteren Zeitpunkt einen Wert zuzuweisen:

```c
  zahl = 42; // Zuweisung eines Wertes
```

*`printf("Die Zahl: %d\n", zahl);`*

Das ist die **Ausgabe**.

`printf` steht für *print formatted* und schreibt Text auf den Bildschirm (genauer: in die Konsole).

Das `%d` ist ein Platzhalter[5] für eine ganze Zahl, die an dieser Stelle ausgegeben wird.

Das `\n` ist ein **Zeilenumbruch**. Es sorgt dafür, dass der Cursor nach der Ausgabe in die nächste Zeile springt.

[2] Eine **Deklaration** ist eine Mitteilung an den Compiler: "Es gibt etwas mit diesem Namen und diesm Typ". Auch ist `int main();` eine Deklaration, allerdings fehlt die Definition oder Implementation, also der Code.

[3] Die **Initialisierung** einer Variable ist die erste Zuweisung eines Wertes. Es ist generell empfehlenswert, jede Variable zu initialisieren (wenn man sich nicht sicher ist, nimmt man die `0`), da ansonsten die Variable einen zufälligen Wert haben kann (siehe dazu Kapitel 4.1).

[4] Variablenamen sind nicht beliebig. Es gibt eine ganze Reihe von Regeln, die man befolgen muss. Für den Anfang sollte folgendes gelten: die Variable beginnt mit einem Buchstaben (groß oder klein), danach folgen Buchstaben (groß und klein), Zahlen und das "underscore" (der Unterstrich _). Wenn Ihr es genauer wissen wollt, dann lest im Anhang nach (Kap. B.2.8).

[5] Um genau zu sein, es ist eine **Formatierungsanweisung**. Eine ausführliche Erklärung findet Ihr im Anhang unter `printf` (libc) auf S. 48.

*Merke:*

- Eine Variable wird deklariert und initialisiert. Dabei wird ihr ein Datentyp zugewiesen.
- `printf()` kann auch den Inhalt von Variable in von Menschen lesbarer Form ausgeben.

## 2.3 Programm: Fibonacci

Die Fibonacci-Zahlen[6] sind eine Folge von Zahlen, bei der jede Zahl die Summe der beiden vorhergehenden ist. Die Folge beginnt mit den Zahlen 0 und 1. Die ersten paar Fibonacci-Zahlen sind:

[6] siehe Wikipedia 2003a

$$
\begin{aligned}
F(0) &= 0,\\
F(1) &= 1,\\
F(2) &= 1,\\
F(3) &= 2,\\
F(4) &= 3,\\
F(5) &= 5,\\
F(6) &= 8,
\end{aligned}
$$

und so weiter. Die Fibonacci-Zahlen haben viele interessante Eigenschaften und Anwendungen in der Mathematik, der Natur und der Informatik. Sie tauchen in der Natur auf, zum Beispiel in der Anordnung von Blättern an einem Stängel, in der Verzweigung von Bäumen, in der Anordnung von Blütenblättern und in der Struktur von Tannenzapfen.

Die Fibonacci-Zahlen werden folgendermaßen berechnet: Die ersten beiden Zahlen der Folge sind 0 und 1. Jede weitere Zahl ist die Summe der beiden vorhergehenden Zahlen. Hier ist ein Beispielprogramm, das die ersten 16 Fibonacci-Zahlen berechnet und ausgibt.

```c
/* Die Fibonacci-Sequenz. */
#include <stdio.h>

int main() {
  int a = 0;
  int b = 1;

  for(int i = 1; i <= 16; i = i+1) {
    int c = a + b;
    a = b;
    b = c;

    printf("Fibonacci %d: %d\n", i, a);
  }
  return 0;
}
```

*Erklärung:*

`int a = 0;`

Wir initialisieren die erste Fibonacci-Zahl mit 0.

`int b = 1;`
Wir initialisieren die zweite Fibonacci-Zahl mit 1.

`for(int i = 1; i <= 16; i = i+1) { … }`
Wir starten eine Schleife, die 16 Mal durchläuft, um die ersten 16 Fibonacci-Zahlen zu berechnen. Seht auch mal den Abschnitt B.2.6 in der Referenz, um mehr über Schleifen zu erfahren.

`int c = a + b;`
Wir berechnen die nächste Fibonacci-Zahl, indem wir die beiden vorhergehenden Zahlen addieren.

`a = b;`
Wir verschieben die Werte: Die aktuelle zweite Zahl wird zur ersten Zahl

`b = c;`
und die neu berechnete nächste Zahl wird zur zweiten Zahl.

`printf("Fibonacci %d: %d\n", i, a);`
Wir geben die aktuelle Fibonacci-Zahl auf dem Bildschirm aus. Das `%d` ist ein Platzhalter für die aktuelle Zahl, die an dieser Stelle ausgegeben wird.

*Merke:*

- Die `for`-Schleife ermöglicht es einem, Code gemäß einer Bedingung beliebig oft zu wiederholen.

## 2.4 Programm: Reihen

Die Zahl $\pi$ ist eine sehr wichtige Zahl in der Mathematik. Sie lässt sich aus einer unendlichen Reihe berechnen:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \ldots$$

Die Reihe geht also unendlich weiter. In der Praxis kann man aber nur endlich viele Terme berechnen. Je mehr Terme man berechnet, desto genauer wird das Ergebnis. Im folgenden Programm wird diese Summe mit 1000000 Schritten berechnet (auf der Konsole wird $\pi$ ausgegeben, indem man obige Summe einfach vervierfacht):

```c
/* Berechnung von Pi/4 aus der Reihe 1 - 1/3 + 1/5 - 1/7 + ... */
#include <stdio.h>

int main() {
    double pi = 1.0;
    for (int i = 1; i <= 1000000; i++) {
        double n = 2*i + 1;     // 3, 5, 7, 9, ...
        n = 1.0 / n;            // 1/n

        // Ausgabe der Zwischenergebnisse
        printf("%9d Pi: %.15f\n", i, 4*pi);

        if (i & 1) // Bedingung: i gerade?
            pi -= n; // ja (true)
        else
            pi += n; // nein (false)
    }
    return 0;
}
```

**Übung 2a:** Einige von Euch haben sich vielleicht schon ein paar Stellen von $\pi$ gemerkt und werden feststellen, dass das Ergebnis sechs Stellen nach dem Komma abweicht. Ihr könnt einmal das `printf()` mit folgernder Anweisung tauschen, die Euch die Abweichung von $\pi$ anzeigt:

```c
printf("%9d Pi: %.15f  (% .15f)\n", i, 4*pi, 4*pi -
    3.14159265358979323846);
```

Wie groß wäre die Abweichung nach 1000 Schritten?

**(Lösung S. 53)**

*Erklärung:*

*`double pi = 1.0;`*

Wir initialisieren die erste Fibonacci-Zahl mit 0. Es handelt sich dabei

um eine Gleitkommazahl, also eine Zahl mit Kommastellen.

*`for (int i = 1; i <= 1000000; i++)`*

Das ist eine Schleifenanweisung, die wir schon im vorhergehenden Kapitel kennengelernt haben. Neu ist hier allerdings, wie *i* gezählt wird. Der Operator `++` sagt, dass *i* um eins erhöht werden soll. Wie er genau funktioniert, werden wir aber später noch kennen lernen.

*`double n = 2*i + 1`*

Oben wurde beschrieben, wie man $\pi$ aus einer Summe berechnen kann. Wenn nochmal schaut, dann sind in der Reihe immer die ungeraden Zahlen 3, 5, 7, usw. Unser Zähler *i* liefert jedoch die Zahlen 1, 2, 3, ... Die Rechnung $2 \cdot i + 1$ liefert nun genau diese ungeraden Zahlen, da wir $\pi$ aber mit einer Gleitkommazahl berechnen muss auch das eine Gleitkommazahl sein.

*`n = 1.0 / n`*

Was wir addieren oder subtrahieren müssen, ist aber nicht 3, 5, 7, usw. sondern 1/3, 1/5, 1/7, usw., wir bilden also hier den Kehrwert $n \leftarrow \frac{1}{n}$ (in Worten: *n* wird der Kehrwert $\frac{1}{n}$ zugeordnet).

*`if (i & 1)`*

Da wir abwechselnd addieren und subtrahieren müssen, muss eine Entscheidung getroffen werden, was von beidem stattfindet. Das macht man mit dem `if`:

```
if (Bedingung) {
    // Befehle, die ausgeführt werden,
    // wenn die Bedingung "wahr" (true) ist.
}
else {
    // Befehle, die ausgeführt werden,
    // wenn die Bedingung "falsch" (false) ist.
}
```

Bemerkt, dass wir die geschweiften Klammern im Code weglassen konnten, weil es nur ein einziger Befehl ist (das gleiche gilt übrigens auch für die `for`-Schleife).

Die Bedingung ist nun `i & 1`. Wenn ihr Euch an das Kapitel über "Bitweise Operationen" erinnert, dann prüft diese Bedingung ob das 0-te Bit eine 1 ist. Das ist genau dann der Fall, wenn *i* ungerade ist. Bildlich sieht das so aus:

```
      i             i & 1
0    00000000    00000000
1    00000001    00000001
```

```
2    00000010    00000000
3    00000011    00000001
4    00000100    00000000
...
```

*Merke:*

- Für Gleitkommazahlen verwenden wir den Datentyp `double`.
- Mit bitweisen Operationen kann man z.B. feststellen, ob eine Zahl gerade oder ungerade ist.

## 2.5 Programm: Lauflicht

Im folgenden C-Programm wird ein sogenanntes “Lauflicht” erzeugt. Dabei wird ein Sternchen von links nach rechts über den Bildschirm bewegt.

Um genau zu sein, was wir machen wollen ist: FIXME

Das Programm läuft dabei unendlich lange, bis es mit `Strg+C` abgebrochen wird.

```c
#include <stdio.h>

int main() {
    while(1) {
        for (int i = 0; i < 20; i++) {
            printf("\r");
            for (int j = 0; j < 20; j++) {
                if (i == j)
                    printf("*");
                else
                    printf(" ");
            }
        }
    }

    return 0;
}
```

Wenn ihr dieses Programm nun ausprobiert, stellt Ihr fest, dass fast nichts passiert. Woran liegt das? FIXME

```c
#define __GNU_SOURCE
#include <unistd.h>
#include <stdio.h>

int main() {
    while(1) {
        for (int i = 0; i < 20; i++) {
            printf("\r");
            for (int j = 0; j < 20; j++) {
                if (i == j)
                    printf("*");
                else
                    printf(" ");
            }
            fflush(stdout);
            usleep(100000);
        }
    }

    return 0;
}
```

# *3*
# *Etwas Physik*

# 4
# Regensimulation in C

## 4.1 Warnungen des Compilers

FIXME

```c
/* Warnungen des Compilers */
#include <stdio.h>

int main() {
  int zahl;
  printf("Die Zahl: %d\n", zahl);
  return 0;
}
```

```
10_warnungen.c: In Funktion »main«:
10_warnungen.c:6:3: Warnung: »zahl« ist nicht initialisiert,
                    wird aber benutzt [-Wuninitialized]
    6 |   printf("Die Zahl: %d\n", zahl);
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
10_warnungen.c:5:7: Anmerkung: »zahl« wurde hier deklariert
    5 |   int zahl;
      |       ^~~~
```

*Erklärung:*

*Merke:*

# Teil II

# Die Programmiersprache C++

# Teil III

# Anhang

# A
# Software

Als Programmierumgebung wurde VSCode verwendet. Zusätzlich ist es empfehlenswert, die Erweiterung “C C++ Toolkit” zu installieren. Diese Erweiterung bietet unter anderem Syntaxhervorhebung, Autovervollständigung und Debugging-Funktionen für C und C++. Die Möglichkeiten, die VScode bietet, sind sehr umfangreich. Daher ist es empfehlenswert, wenn ihre jemanden fragt, der sich mit VScode auskennt, um die Umgebung optimal einzurichten.

# *B*
# *Leitfaden*

## *B.1 Tastenkombinationen*

Tastaturen haben ein paar besondere Tasten, die es einem erlauben, zum Beispiel Text zu markieren, kopieren und einzufügen. Auch lassen sich dadurch einige spezielle Zeichen, die oft bei der Programmierung verwendet werden, die auf einer deutschen Tastatur keine eigenen Tasten belegen, eingeben. Die Tasten, die ihr dafür benötigt, sind:

*Shift* Die sogenannte "Umschalttaste". [1]Sie schaltet zwischen Klein- und Großschreibung, sowie zwischen den verschiedenen abgebildeten Symbolen um.

*Feststelltaste* Im Englischen "Caps lock", sie befindet oberhalb der "Shift"-Taste, sie ist dafür gedacht, dauerhaft zwisch Groß- und Kleinbuchstaben umzuschalten.

*Strg* Die Steuerungstaste (im Englischen "Control" bzw. abgekürzt "Ctrl").

*Alt* "Alt" steht für "Alternative". Sie wird häufig auch in Verbindung mit "Shift" und "Strg" verwendet. Allein wird sie z.B. verwendet, um zwischen Fenstern mit "Alt+Tab" ("Tab" siehe weiter unten) hin- und herzuschalten.

*AltGr* "AltGr" steht für "Alternative Grafik", ist im Prinzip die "rechte Alt-Taste", hat aber auch die Funktion, Zeichen einzugeben, die auf, z.B. einer deutschen Tastatur, sonst nicht zur Verfügung stehen.

*Esc* Das ist die sogenannte "escape"-Taste, auf Deutsch übersetzt "Flucht". Mit dieser Taste lassen sich viele Vorgänge abbrechen.

*Cursor* Damit sind die Tasten gemeint, mit den man nach rechts, links, oben unten geht. Im Deutschen werden sie auch "Kursor", "Schreibmarke", "Eingabezeiger" usw. bezeichnet, oder einfach als "Pfeiltasten".

*Tab* Die "Tabulator"-Taste dient dazu, Text richtig einzurücken.

[1] Sucht mal auf `https://youtube.com` nach «Umschalttaste mechanische Schreibmaschine», wenn ihr wissen wollt, woher dieser Begriff kommt.

### *B.1.1 Sonderzeichen*

Die folgenden Sonderzeichen gelten für eine deutsche Tastatur:

| **Tastenkombination** | **Zeichen** | **Bedeutung** |
|---|---|---|
| AltGr + Q | @ | At-Zeichen |
| AltGr + 7 | { | Geschweifte Klammer auf |
| AltGr + 8 | [ | Eckige Klammer auf |
| AltGr + 9 | ] | Eckige Klammer zu |
| AltGr + 0 | } | Geschweifte Klammer zu |
| AltGr + ß | \ | Backslash |
| AltGr + < | \| | Pipe |
| AltGr + 3 | # | Hashtag |
| AltGr + + | ~ | Tilde |

### *B.1.2 Referenz*

Die folgende Liste von Tastenkombination gilt für viele, aber nicht für alle Programme:

| **Tastenkombination** | **Bedeutung** |
|---|---|
| *Markieren* | |
| Esc | Markierung aufheben |
| Shift+Cursor | Text markieren (alternativ mit Maus) |
| Strg+A | Alles auswählen (“select all”) |
| *Kopieren und Einfügen* | |
| Strg+C | Markierten Text kopieren |
| Strg+X | Markierten Text ausschneiden |
| Strg+V | Text einfügen |
| Strg+Z | Eingabe rückgängig machen (“undo”) |
| *Häufige Funktionen* | |
| Strg+S | Speichern (immer wieder drücken!) |
| Strg+F | Suchen (“find”) |
| Strg+H | Suchen und ersetzen (“replace”) |

## B.2 C

### B.2.1 Grundlagen

### B.2.2 Datentypen

#### Ganzzahlen

C und C++ bieten verschiedene Datentypen für Ganzzahlen an, darunter `int`, `short`, `long` und `long long`. Jeder dieser Typen hat eine unterschiedliche Größe und einen unterschiedlichen Wertebereich. Die Größe hängt von der Architektur des Computers ab, auf dem das Programm ausgeführt wird. Diese sind alle **vorzeichenbehaftet**, was bedeutet, dass sie sowohl positive als auch negative Werte darstellen können. Es gibt auch die vorzeichenlosen Varianten `unsigned int`, `unsigned short`, `unsigned long` und `unsigned long long`, die nur nicht-negative Werte darstellen können (vergleicht das nochmal mit dem Abschnitt 1).

Die Liste der ganzzahligen Datentypen in C und C++ ist wie folgt:

*`int`*
: Standard-Ganzzahltyp, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: -2.147.483.648 bis 2.147.483.647 (vorzeichenbehaftet) oder 0 bis 4.294.967.295 (vorzeichenlos).

*`short`*
: Kürzere Ganzzahl, in der Regel 2 Bytes (16 Bit) groß, Wertebereich: -32.768 bis 32.767 (vorzeichenbehaftet) oder 0 bis 65.535 (vorzeichenlos).

*`long`*
: Längere Ganzzahl, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: -2.147.483.648 bis 2.147.483.647 (vorzeichenbehaftet) oder 0 bis 4.294.967.295 (vorzeichenlos).

*`long long`*
: Noch längere Ganzzahl, in der Regel 8 Bytes (64 Bit) groß, Wertebereich: -9.223.372.036.854.775.808 bis 9.223.372.036.854.775.807 (vorzeichenbehaftet) oder 0 bis 18.446.744.073.709.551.615 (vorzeichenlos).

#### Gleitkommazahlen

Gleitkommazahlen, also Zahlen mit Nachkommastellen, werden in C und C++ durch die Datentypen `float`, `double` und `long double` dargestellt. Diese Typen unterscheiden sich in der Genauigkeit und dem Wertebereich. Die Liste der Gleitkomma-Datentypen in C und C++ ist wie folgt:

*`float`*
: Einfachpräzisions-Gleitkommazahl, in der Regel 4 Bytes (32 Bit) groß, Wertebereich: ca. 1.2E-38 bis 3.4E+38

*`double`*
: Doppelpräzisions-Gleitkommazahl, in der Regel 8 Bytes (64 Bit) groß, Wertebereich: ca. 2.3E-308 bis 1.7E+308

*`long double`*
: Erweiterte Präzisions-Gleitkommazahl, in der Regel 10, 12 oder 16 Bytes (80, 96 oder 128 Bit) groß, Wertebereich: abhängig von der Implementierung, aber in der Regel größer als `double`

Hier wird eine eigenartige Darstellung von Gleitkommazahlen verwendet, die als **IEEE 754 Standard** bekannt ist. Diese Darstellung ermöglicht es, eine große Bandbreite von Zahlen darzustellen. Das "E" steht für "Exponent" und zeigt an, dass die Zahl in wissenschaftlicher Notation dargestellt wird. Zum Beispiel bedeutet `1.2E-38` die Zahl $1.2 \cdot 10^{-38}$, also eine sehr kleine Zahl, während `3.4E+38` die Zahl $3.4 \cdot 10^{38}$ ist, also eine sehr große Zahl. Man kann sagen, dass die Zahl nach "E" die Zahl der Stellen angibt, um die das Komma verschoben werden muss, um die Zahl in Standardform zu bringen. Ein negativer Exponent bedeutet, dass das Komma nach links verschoben wird, während ein positiver Exponent bedeutet, dass das Komma nach rechts verschoben wird. Damit wird, z.B. die Zahl `1.23E+3` zu 1230 und die Zahl `4.56E-2` zu 0.0456.

### *Zeichen und Zeichenketten*

Zeichen werden in C und C++ durch den Datentyp `char` dargestellt. Eine Zeichenvariable speichert ein einzelnes Zeichen und belegt in der Regel 1 Byte.[2] Zeichenketten sind Arrays von Zeichen, die durch das Nullzeichen `'\0'` terminiert sind. Sie werden durch den Datentyp `char[]` oder `char*` dargestellt.

[2] Bei den Prozessoren, die heutzutage gebaut werden, kann man sich darauf verlassen, dass ein `char` immer 1 einzelnes Byte belegt. Dies war aber in der Geschichte der Computer nicht immer der Fall. Das genau zu beschreiben ist aber ein sehr kompliziertes Thema und eher geeignet für einen Fortgeschrittenenkurs.

### *`void`*

Der Datentyp `void`, was so viel wie "Leere" oder "Lücke" bedeutet, wird verwendet, um anzugeben, dass eine Funktion keinen Rückgabewert hat. Er kann auch in Zeigern verwendet werden, um einen Zeiger auf einen unbekannten Datentyp zu deklarieren.

### *Zeiger*

Zeiger sind Speicheradressen.

### *`const`*

Eine der schwierigsten Themen in C und C++ ist die Verwendung von `const`.

### *B.2.3 Strukturen*

Um komplexere Daten zu speichern, können wir in C und C++ **Strukturen** verwenden. Eine Struktur ist eine Sammlung von Variablen, die unter einem gemeinsamen Namen zusammengefasst sind. Jede Variable innerhalb einer Struktur wird als **Mitglied** bezeichnet. Strukturen ermöglichen es uns, verschiedene Datentypen in einer einzigen Einheit zu kombinieren. Hier ist ein Beispiel für eine Struktur, die einen Punkt im zweidimensionalen Raum darstellt:

```
struct Point {
    int x; // x-Koordinate
    int y; // y-Koordinate
};
```

Wir können dann eine Variable vom Typ `Point` erstellen und auf ihre Mitglieder zugreifen:

```
struct Point p1; // Erstellen einer Point-Variable
p1.x = 10; // Setzen der x-Koordinate
p1.y = 20; // Setzen der y-Koordinate
printf("Punkt: (%d, %d)\n", p1.x, p1.y); // Ausgabe
```

Alternativ können wir auch die Struktur direkt bei der Deklaration initialisieren:

```
struct Point p2 = {30, 40}; // Erstellen mit Initialisierung
printf("Punkt: (%d, %d)\n", p2.x, p2.y); // Ausgabe
```

### *B.2.4 Anweisungsblock*

Ein Anweisungsblock (englisch: *block*) ist eine Gruppe von Anweisungen, die zusammengefasst und als eine Einheit behandelt werden. In C und C++ wird ein Anweisungsblock durch geschweifte Klammern `{ ... }` definiert. Ein Anweisungsblock kann überall dort verwendet werden, wo eine einzelne Anweisung erwartet wird, z.B. in Kontrollstrukturen wie `if`, `for` oder `while`. Innerhalb eines Anweisungsblocks können Variablen deklariert werden, die nur innerhalb dieses Blocks sichtbar sind (lokale Variablen). Dies ermöglicht eine bessere Organisation des Codes und verhindert Namenskonflikte.

Blöcke können auch verschachtelt werden, d.h. ein Anweisungsblock kann innerhalb eines anderen Anweisungsblocks enthalten sein. Dies ist besonders nützlich, um den Gültigkeitsbereich von Variablen zu begrenzen und die Lesbarkeit des Codes zu verbessern.

Aufpassen solltet ihr jedoch mit der Benennung von Variablen in verschachtelten Blöcken. Wenn ihr in einem inneren Block eine Variable mit dem gleichen Namen wie in einem äußeren Block deklariert, wird

die äußere Variable innerhalb des inneren Blocks ”verdeckt”(englisch: *shadowed*). Das bedeutet, dass ihr innerhalb des inneren Blocks nur auf die innere Variable zugreifen könnt, während die äußere Variable vorübergehend nicht zugänglich ist. Dies kann zu Verwirrung führen und sollte vermieden werden, indem ihr eindeutige Namen für Variablen verwendet.Beispiel:

```
int main() {
    int x = 10; // äußere Variable
    {
        int x = 20; // innere Variable verdeckt die Äußere
        printf("Innere x: %d\n", x); // Ausgabe: 20
    }
    printf("Äußere x: %d\n", x); // Ausgabe: 10
    return 0;
}
```

Es sollte angemerkt werden, dass moderne C- und C++-Compiler in der Regel Warnungen ausgeben, wenn Variablen in verschachtelten Blöcken verdeckt werden. Es ist eine gute Praxis, eindeutige Namen für Variablen zu verwenden, um Verwirrung zu vermeiden.

### *B.2.5 Kontrollstrukturen*

Sowohl in C als auch in C++ gibt es verschiedene Kontrollstrukturen, die den Ablauf eines Programms steuern. Dazu gehören bedingte Anweisungen wie `if`, `else if` und `else`, sowie Schleifen im folgenden Abschnitt. Diese Strukturen ermöglichen es, Entscheidungen zu treffen und bestimmte Codeabschnitte wiederholt auszuführen, basierend auf Bedingungen oder Zählvariablen.

*`if`*

Die `if`-Anweisung wird verwendet, um eine Bedingung zu überprüfen und einen Codeblock auszuführen, wenn die Bedingung wahr ist. Die Syntax lautet:

```
if (Bedingung) {
    // Code, der ausgeführt wird, wenn die Bedingung wahr ist
}
```

*`else if`*

Die `else if`-Anweisung wird verwendet, um eine weitere Bedingung zu überprüfen, wenn die vorherige `if`-Bedingung falsch war. Die Syntax lautet:

```
if (Bedingung1) {
    // Code, der ausgeführt wird, wenn Bedingung1 wahr ist
} else if (Bedingung2) {
```

```
    // Code, der ausgeführt wird, wenn Bedingung2 wahr ist
}
```

*else*
: Die else-Anweisung wird verwendet, um einen Codeblock auszuführen, wenn keine der vorherigen Bedingungen wahr ist. Die Syntax lautet:

```
if (Bedingung1) {
    // Code, der ausgeführt wird, wenn Bedingung1 wahr ist
} else if (Bedingung2) {
    // Code, der ausgeführt wird, wenn Bedingung2 wahr ist
} else {
    // Code, der ausgeführt wird, wenn keine der Bedingungen wahr ist
}
```

Eine **Bedingung** ist ein Ausdruck, der entweder `true` (wahr) oder `false` (falsch) ergibt. In C und C++ werden Bedingungen durch Vergleiche oder sogenannte logische Ausdrücke gebildet.

Ein Vergleich bildet sich aus zwei Werten und einem Vergleichsoperator. Wenn wir zum Beispiel feststellen wollen ob zwei Zahlen `a` und `b` gleich sind, dann schreiben wir `a == b`. Dabei wird `==` als Operator bezeichnet (um genau zu sein, als Vergleichsoperator).

Liste aller Vergleichsoperatoren: Mögliche Vergleiche sind:

`==`
: Gleichheit: Überprüft, ob zwei Werte gleich sind.

`!=`
: Ungleichheit: Überprüft, ob zwei Werte ungleich sind.

`<`
: Kleiner als: Überprüft, ob der linke Wert kleiner als der rechte Wert ist.

`>`
: Größer als: Überprüft, ob der linke Wert größer als der rechte Wert ist.

`<=`
: Kleiner oder gleich: Überprüft, ob der linke Wert kleiner oder gleich dem rechten Wert ist.

`>=`
: Größer oder gleich: Überprüft, ob der linke Wert größer oder gleich dem rechten Wert ist.

Logische Operatoren werden verwendet, um mehrere Bedingungen zu kombinieren oder zu negieren. Die wichtigsten logischen Operatoren in C und C++ sind:

*&&*
: Logisches UND: Gibt `true` zurück, wenn beide Bedingungen wahr sind.

*||*
: Logisches ODER: Gibt `true` zurück, wenn mindestens eine der Bedingungen wahr ist.

*!*
: Logisches NICHT: Negiert die Bedingung, d.h. gibt `true` zurück, wenn die Bedingung falsch ist, und `false` zurück, wenn die Bedingung wahr ist.

### *B.2.6 Schleifen*

C und C++ bieten drei Arten von Schleifen an: `for`, `while` und `do-while`. Jede Schleife hat ihre eigenen Anwendungsfälle und Syntax.

*`for`-Schleife*
: Die `for`-Schleife wird verwendet, wenn die Anzahl der Schritte im Voraus bekannt ist. Sie hat die folgende Syntax:

```
for(Initialisierung; Bedingung; Inkrement) {
    // Code, der wiederholt ausgeführt wird
}
```

*`while`-Schleife*
: Die `while`-Schleife wird verwendet, wenn die Anzahl der Schritte nicht im Voraus bekannt ist und die Schleife so lange ausgeführt werden soll, wie eine bestimmte Bedingung erfüllt ist. Sie hat die folgende Syntax:

```
while(Bedingung) {
    // Code, der wiederholt ausgeführt wird
}
```

*`do-while`-Schleife*
: Die `do-while`-Schleife ist ähnlich wie die `while`-Schleife, aber der Code innerhalb der Schleife wird mindestens einmal ausgeführt, bevor die Bedingung überprüft wird. Sie hat die folgende Syntax:

```
do {
    // Code, der wiederholt ausgeführt wird
} while(Bedingung);
```

Die `for`-Schleife nimmt hier eine besondere Stellung ein. Am besten erklärt man die `for`-Schleife, indem man sie in eine `while`-Schleife umwandelt. Die `for`-Schleife

```
for(Initialisierung; Bedingung; Inkrement) {
    // Code, der wiederholt ausgeführt wird
}
```

ist äquivalent zu:

```
{
    Initialisierung;
    while(Bedingung) {
        // Code, der wiederholt ausgeführt wird
        Inkrement;
    }
}
```

Die **Initialisierung** wird nur einmal zu Beginn der Schleife ausgeführt. Sie dient gewöhnlich dazu, eine Zählvariable zu deklarieren und zu initialisieren. Die **Bedingung** wird vor jedem Durchlauf überprüft, und das **Inkrement** wird am Ende jedes Durchlaufs ausgeführt (Inkrement heißt wörtlich Ërhöhung"). Wenn wir zum Beispiel 1 bis 10 zählen wollen, dann schreiben wir:

```
for(int i = 1; i <= 10; i++) {
    printf("%d\n", i);
}
```

oder von 100 in 10-er Schritten bis 0 zählen wollen, dann schreiben wir:

```
for(int i = 100; i >= 0; i -= 10) {
    printf("%d\n", i);
}
```

### *B.2.7 Funktionen*

Um ein Programm in kleinere, wiederverwendbare Teile zu zerlegen, können wir in C und C++ **Funktionen** verwenden. Eine Funktion ist ein benannter Codeblock, der eine bestimmte Aufgabe ausführt und optional Werte zurückgeben kann. Funktionen ermöglichen es uns, den Code zu organisieren, die Lesbarkeit zu verbessern und die Wiederverwendbarkeit zu erhöhen.

Eine Funktion besteht aus einem Funktionskopf, der den Namen der Funktion, die Parameter und den Rückgabewert definiert, und einem Funktionskörper, der die Anweisungen enthält, die ausgeführt werden, wenn die Funktion aufgerufen wird. Die Syntax für die Definition einer Funktion lautet:

```
Rückgabewert Funktionsname(Parameterliste) {
    // Anweisungen
}
```

Der **Rückgabewert** gibt den Datentyp des Wertes an, den die Funktion zurückgibt (siehe oben B.2.2). Wenn die Funktion keinen Wert zurückgibt, wird der Rückgabewert als `void` angegeben. Die **Parameterliste** enthält die Eingabewerte, die an die Funktion übergeben werden, und kann leer sein, wenn keine Eingaben erforderlich sind. Ein Parameter, häufig auch als Argument bezeichnet, ist eine Variable, die innerhalb der Funktion verwendet wird und deren Wert beim Aufruf der Funktion übergeben wird. Ein Argument setzt sich aus Datentype und Variablendamen zusammen. Die Parameterliste kann mehrere Parameter enthalten, die durch Kommas getrennt sind.

Zum Beispiel definieren wir eine Funktion, die zwei ganze Zahlen addiert und das Ergebnis zurückgibt:

```
int addiere(int a, int b) {
    return a + b; // Rückgabe der Summe
}
```

Wir können diese Funktion dann in unserem Hauptprogramm aufrufen:

```
int main() {
    int ergebnis = addiere(5, 3); // Aufruf der Funktion
    printf("Ergebnis: %d\n", ergebnis); // Ausgabe
    return 0;
}
```

### *B.2.8 Namen*

### *B.2.9 Referenz*

**`fflush`**

**Syntax**:

```
#include <stdio.h>

int fflush(FILE *stream);
```

**Beschreibung**:
FIXME

**Rückgabewert**:
War der Funktionsaufruf erfolgreich, wird 0 zurückgegeben, `EOF` jedoch wenn ein Fehler aufgetreten ist.

### `printf`

**Syntax**:

```
#include <stdio.h>

int printf(const char *format, ...);
```

**Beschreibung**:
`printf()` schreibt formattierten Text in die Standardausgabe (stdout).
Der `format`-String enthält:

- Normale Zeichen (werden direkt ausgegeben)
- Format-Spezifizierungen (beginnen mit `%`)

**Die wichtigsten Format-Spezifizierungen:**

| | |
|---|---|
| `%d` | Ganzzahl (dezimal) |
| `%f` | Gleitkommazahl |
| `%c` | Zeichen (char) |
| `%s` | Zeichenkette (String) |
| `%x` | Hexadezimal (klein) |
| `%X` | Hexadezimal (groß) |
| `%p` | Zeiger (Pointer) |

**Beispiel**:

```c
#include <stdio.h>

int main() {
    int zahl = 42;
    printf("Die Zahl ist: %d\n", zahl);
    printf("Adresse: %p\n", (void*)&zahl);
    return 0;
}

```

**Rückgabewert**:
Die Anzahl der ausgegebenen Zeichen, oder `-1` bei Fehler.

**Siehe auch**:
`scanf` (libc), `sprintf` (libc), `fprintf` (libc)

### stdin, stdout, stderr

**Syntax**:

```
#include <stdio.h>

extern FILE *stdin;
extern FILE *stdout;
extern FILE *stderr;
```

**Beschreibung**:
Unter normalen Bedingungen hat jedes UNIX-Betriebssystem drei Kanäle offen, die es erlauben, Daten mit einem Benutzer auszutauschen, nämlich

- den Eingabekanal (z.B. für Tastatureingabe),
- den Ausgabekanal (z.B. für Textausgabe auf der Konsole) und
- den Fehlerkanal (der nur für die Ausgabe von Fehlertexten bestimmt ist, meistens auch auf der Konsole).

FIXME

### usleep

**Syntax**:

```
#define __GNU_SOURCE
#include <unistd.h>

int usleep(useconds_t usec);
```

**Beschreibung**:
`usleep()` unterbricht die Ausführung des Programms mindestens `usec` Mikrosekunden. Die genau Wartezeit kann jedoch mehr sein, wenn der Computer sehr viel zu tun hat.

**Rückgabewert**:
War der Funktionsaufruf erfolgreich, wird 0 zurückgegeben, `-1` jedoch wenn ein Fehler aufgetreten ist.

# C
# *Lösungen zu den Übungen*

**Lösung 1a**
Die vollständige Wahrheitstabelle für die Addition von zwei Bits und den Übertrag sieht folgendermaßen aus:

| A | B | S | C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Wenn Ihr Euch die Wahrheitstabellen für `AND` und `XOR` in Kapitel 1.2 nochmal anschaut, dann könnt Ihr sehen, dass die Summe S mit der `xor`-Operation berechnet werden kann und der Übertrag C mit der `and`-Operation.

**Lösung 1b**
Vermutlich wird Euch schon einmal jemand gesagt haben, dass man bei der Lösung einer Aufgabe alle Zwischenschritte aufschreiben soll. Wenn der Autor ehrlich ist: Er hat selbst schon Punkte in Klassenarbeiten verloren, weil er die Zwischenschritte nicht aufgeschrieben hat. Das war ärgerlich – und hätte sich mit einem klaren Plan vermeiden lassen. Das ist aber genau der Punkt bei der Lösung der Aufgabe: ein klarer Plan (oder, wie die Informatiker sagen: ein Algorithmus).

In der Aufgabe wurden die ersten Schritte bereits vorgegeben:

```
S = A xor B
C = A and B
```

Das sieht zunächst folgendermaßen aus:

```
A  00100111       39
B  00000110     +  6
   --------
S  00100001       33
C  00000110        6
```

Das macht jetzt natürlich noch keine Sinn, denn wir wissen ja, dass der *Übertrag* nicht an der gleichen Stelle wie die Summe steht, sondern zum nächsten Bit weiter links gehört. Das bedeutet, dass wir den Übertrag um eine Stelle nach links verschieben müssen:

```
A     00100111       39
B     00000110     +  6
      --------
S     00100001       33
C<<1  00001100       12
```

Auf den ersten Blick sieht das jetzt schon besser aus, aber wir wissen ja, dass wir die Summe und den Übertrag noch einmal addieren müssen. Das bedeutet, dass wir die beiden Zahlen $S$ und $C << 1$ wieder addieren müssen. Das können wir wieder mit den gleichen Schritten tun:

```
S = S xor (C<<1)
C = S and (C<<1)
```

Das sieht dann folgendermaßen aus:

```
S     00100001       33
C<<1  00001100       12
      --------
S     00101101       45
C<<1  00000000        0
```

Jetzt ist der Übertrag $C$ gleich 0, das bedeutet, dass wir die Summe berechnet haben. Die Summe von $A + B$ ist also $S = 00101101$ oder dezimal 45.

Ihr fragt Euch jetzt vielleicht, warum jetzt nochmal eine Addition? Im Kopf macht ihr das mit zweistelligen Zahlen ja auch nicht mehr. Aber schauen wir uns den Algorithmus noch einmal an, indem wir 7 und 3 addieren:

```
A     00000111        7
B     00000011      + 3
      --------
S     00000100        4    Schritt 1
C<<1  00000110        6
      --------
S     00000010        2    Schritt 2
C<<1  00001000        8
```

Jetzt könntet Ihr vermutlich denken, dass wir den Schritt der Berechnung von S und C so lange wiederholen müssen, bis der Übertrag gleich 0 ist, aber tatsächlich ist das nicht notwendig, so dass wir nur noch einmal die beiden Schritte ausführen müssen.

```
S = A xor B
C = A and B
```

```
S = S xor (C<<1)
C = S and (C<<1)

S = S xor (C<<1)
```

Bemerkt hier, dass der Übertrag $C$ in der letzten Zeile nicht mehr berechnet wird, da wir ja wissen, dass er 0 ist (eigentlich müsste man das mathematisch beweisen, aber das lassen wir hier weg). Das bedeutet, dass wir die Summe von $A+B$ mit bitweisen Operation in fünf Schritten berechnen können.[1]

[1] In der Praxis ist es allerding so, dass man auf elektronischer Ebene, also mithilfe von Logikgattern, die Addition in einem Schritt berechnet. Wenn Ihr Zeit habt, dann sucht mal im Internet nach dem Begriff **Volladdierer**.

**Lösung 2a**

Ab der 7. Programmzeile wird

```
1 double n = 2*i + 1
2 n = 1.0/n;
```

berechnet. Dieses $n$ wird dann zu dem Ergebnis hinzugezäht oder abgezogen – damit ist die Abweichung für $i = 1000$:

$$\Delta\pi = 4\frac{1}{2n+1} = \frac{4}{2001} \approx 0,002\,.$$

# *Literatur*

Wikipedia (2003a). *Fibonacci-Folge*. URL: `https://de.wikipedia.org/wiki/Fibonacci-Folge` (besucht am 24. 07. 2026) (siehe S. 21).

— (2003b). *Hallo-Welt-Programm*. URL: `https://de.wikipedia.org/wiki/Hallo-Welt-Programm` (besucht am 24. 07. 2026) (siehe S. 17).

# *Index*


Binärsystem, 13

C-Funktion
  `fflush`, 47
  `printf`, 48
  `stderr`, 49
  `stdin`, 49
  `stdout`, 49
  `usleep`, 49
C-Makro
  `__GNU_SOURCE`, 27

Lizenz, 2, 7